

GenAI Banking Policy & Compliance Copilot

1. Introduction

1.1 Purpose

The purpose of this system is to build an AI-powered compliance assistant that enables users to query banking policy documents using natural language and receive accurate, context-grounded answers with proper source citations.

1.2 Scope

The system is designed to:

- Accept PDF-based banking policy documents
- Process and index uploaded documents
- Perform semantic search using embeddings
- Generate responses using a local Large Language Model (Phi-2)
- Ensure explainability, traceability, and compliance accuracy

1.3 Definitions

Term	Meaning
RAG	Retrieval-Augmented Generation
Embedding	Vector representation of text
Vector Store	Storage mechanism for embeddings
Chunking	Splitting text into smaller parts

2. System Overview

The system consists of three core modules:

- **Ingestion Module:** Responsible for PDF processing
- **Retrieval Module:** Performs semantic search over indexed data
- **Generation Module:** Generates responses using an LLM

3. System Architecture

The system follows a modular **Retrieval-Augmented Generation (RAG)** architecture and operates entirely on **local hardware** to ensure data privacy and regulatory compliance.

Key Components

User Interface (Streamlit)

- Entry point for users
- Allows PDF upload and query input
- Displays chat history, answers, and source citations

Document Processor

- Extracts text from PDFs
- Cleans and normalizes content
- Splits text into chunks of approximately **512 tokens with 64-token overlap**

Vector Store

- Stores document chunks as vectors
- Uses cosine similarity for retrieval
- Operates as a local semantic search engine

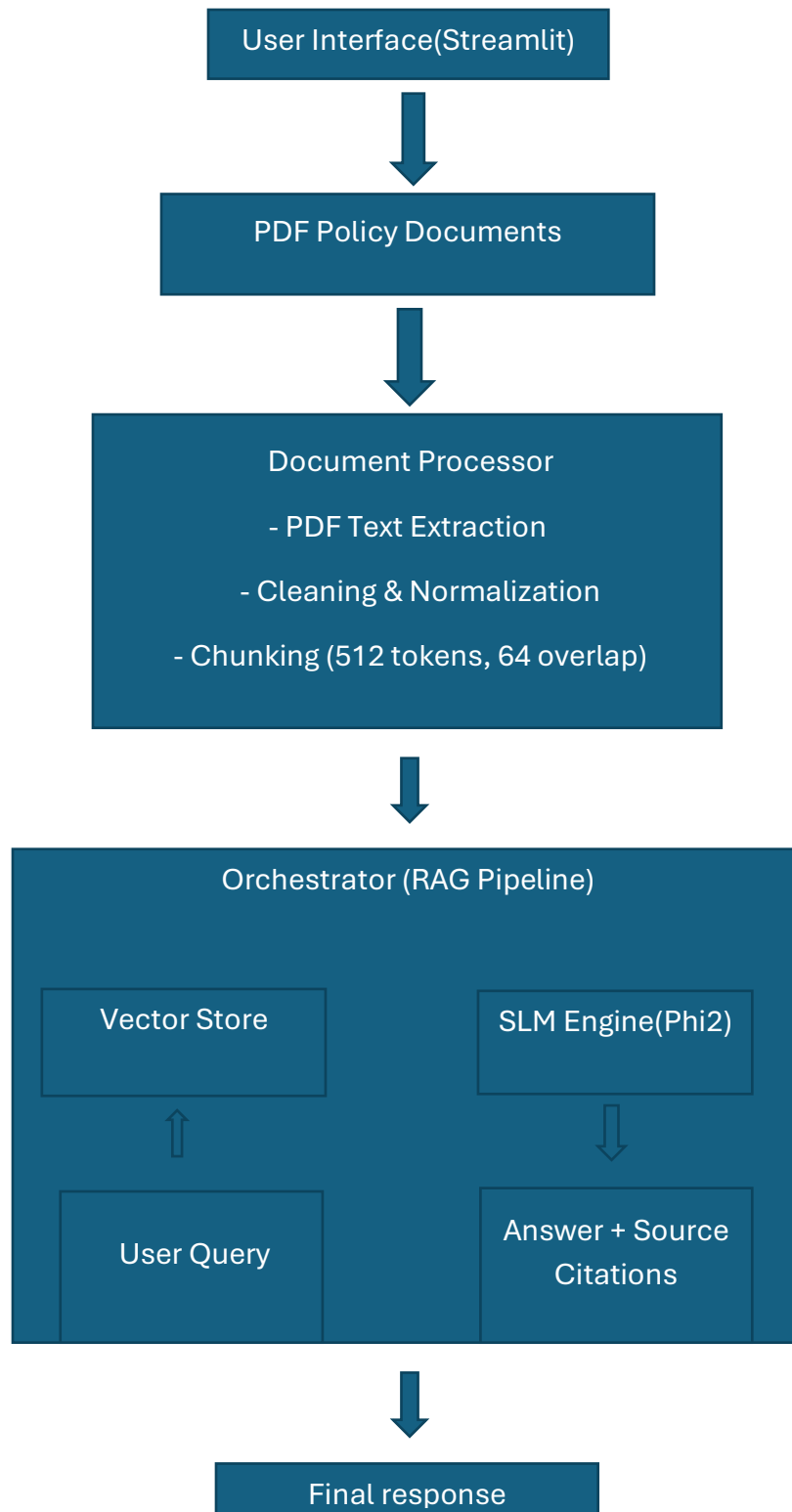
SLM Engine (Phi-2)

- Local deployment of Microsoft Phi-2
- Generates concise, grounded responses using retrieved context

Orchestrator (RAG Pipeline)

- Coordinates data flow between ingestion, retrieval, and generation modules

Diagram



4. Functional Requirements (FR)

4.1 User Interface Requirements

- **FR-UI-01:** Allow users to upload one or more PDF documents
- **FR-UI-02:** Display success/failure messages and indexed documents
- **FR-UI-03:** Accept user queries in natural language
- **FR-UI-04:** Display generated answers with source citations and relevance scores

4.2 Document Processing Requirements

- **FR-DOC-01:** Extract text page-wise from PDFs
- **FR-DOC-02:** Clean noise, normalize formatting, and handle encoding issues
- **FR-DOC-03:** Split text into ~512 token chunks with 64-token overlap
- **FR-DOC-04:** Attach metadata (document name, page number, chunk index) to each chunk

4.3 Embedding & Storage Requirements

- **FR-EMB-01:** Use Sentence Transformers for vector generation
- **FR-EMB-02:** Store embeddings along with chunk text and metadata
- **FR-EMB-03:** Persist data locally across sessions

4.4 Retrieval Requirements

- **FR-RET-01:** Convert user queries into embeddings
- **FR-RET-02:** Use cosine similarity for semantic matching
- **FR-RET-03:** Retrieve top-K most relevant chunks
- **FR-RET-04:** Combine retrieved chunks into structured context

4.5 Generation Requirements

- **FR-GEN-01:** Construct prompts using retrieved context and user query
- **FR-GEN-02:** Generate responses using Phi-2
- **FR-GEN-03:** Restrict responses strictly to retrieved context
- **FR-GEN-04:** Provide final answer with citations and excerpts

4.6 Error Handling Requirements

- **FR-ERR-01:** Return retrieved chunks if SLM generation fails
- **FR-ERR-02:** Prompt user if query input is empty
- **FR-ERR-03:** Display message when no relevant results are found

4.7 Security Requirements

- **FR-SEC-01:** Operate fully offline
- **FR-SEC-02:** No external API or cloud calls

5. Non-Functional Requirements (NFR)

- **Performance:** Low-latency retrieval and response generation
- **Scalability:** Support growing document sets and future FAISS integration
- **Reliability:** Graceful degradation and fallback mechanisms
- **Usability:** Simple, intuitive chat-based interface
- **Security & Privacy:** All data remains local
- **Maintainability:** Modular, well-separated code structure
- **Portability:** Compatible with Windows and Linux
- **Availability:** Fully functional without internet connectivity

6. Implementation Challenges

- **Hardware Constraints:** Running Phi-2 locally requires sufficient RAM and VRAM
- **Strict Offline Operation:** Preventing unintended external calls from libraries
- **Retrieval Precision:** TF-IDF lacks deep semantic understanding, requiring query expansion
- **Hallucination Prevention:** Enforcing strict context-based prompting

7. Key Learnings

- **Efficiency of Small Models:** Phi-2 performs well for focused RAG use cases
- **Context Quality Matters:** Clean chunking and accurate retrieval are critical
- **Transparency Builds Trust:** Citations and relevance scores are essential in compliance
- **Modular Design Advantages:** Easier testing, upgrades, and maintenance

8. Use Case

Actor: User

Flow:

1. User uploads PDF document
2. System processes and indexes document
3. User submits a query
4. System retrieves relevant chunks
5. System generates an answer
6. System displays answer with citations

9. Constraints

- Requires local hardware for model execution
- Works only with text-based PDFs
- Performance depends on system resources

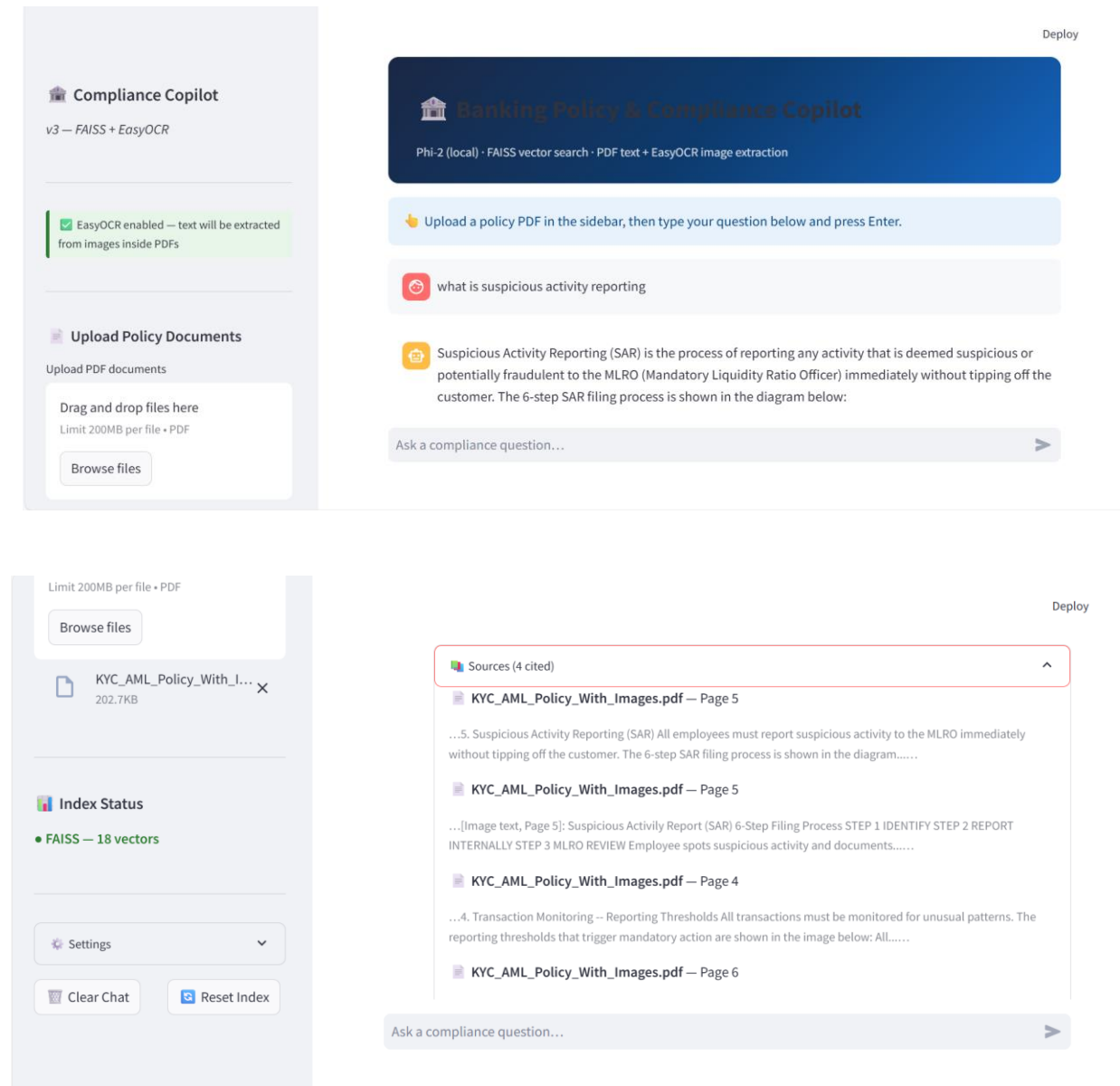
10. Future scope

- Better LLM integration beyond phi2 for example LLaMA
- Adding Multi-Language Support
- Enterprise-Grade Security like data encryption

11. Conclusion

The GenAI Banking Policy & Compliance Copilot delivers a **privacy-preserving, explainable, and efficient AI solution** for regulatory and compliance workflows. By leveraging local RAG architecture, it reduces manual effort, improves accuracy, and ensures full traceability—making it well-suited for sensitive banking environments.

12. Implementation Screenshots



13 Code Snippets

```
"""
config.py - Central configuration for the Compliance Copilot v3.
"""
import os
```

GenAI Banking Policy & Compliance Copilot

```
from pathlib import Path

# — Block ALL HuggingFace network calls —————
os.environ["TRANSFORMERS_OFFLINE"] = "1"
os.environ["HF_DATASETS_OFFLINE"] = "1"
os.environ["HF_HUB_OFFLINE"] = "1"

try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    pass

# — Paths —————
BASE_DIR = Path(__file__).resolve().parent.parent
UPLOAD_DIR = Path(os.getenv("UPLOAD_PATH", str(BASE_DIR / "data" /
"uploads")))
VECTORSTORE_DIR = Path(os.getenv("VECTORSTORE_PATH", str(BASE_DIR / "data" /
"vectorstore")))

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
VECTORSTORE_DIR.mkdir(parents=True, exist_ok=True)

# — Phi-2 Local Path —————
# !! SET THIS TO YOUR LOCAL PHI-2 FOLDER !!
# Example: r"D:\phi-2" or r"C:\Users\Akshay\models\phi-2"
PHI2_MODEL_NAME = os.getenv("PHI2_MODEL_NAME", r"D:\phi-2")

# — Chunking —————
CHUNK_SIZE = int(os.getenv("CHUNK_SIZE", "512"))
CHUNK_OVERLAP = int(os.getenv("CHUNK_OVERLAP", "64"))

# — Retrieval —————
TOP_K_RESULTS = int(os.getenv("TOP_K_RESULTS", "4"))

# — Generation —————
MAX_NEW_TOKENS = int(os.getenv("MAX_NEW_TOKENS", "256"))
TEMPERATURE = float(os.getenv("TEMPERATURE", "0.1"))

# — EasyOCR —————
# Languages for EasyOCR. 'en' = English. Add more if needed e.g. ['en', 'hi']
# Models are downloaded once on first use (~100 MB) and cached locally.
```


GenAI Banking Policy & Compliance Copilot

```
EASYOCR_LANGUAGES = ["en"]
```

```
# — UI —  
APP_TITLE = os.getenv("APP_TITLE", "Banking Policy & Compliance Copilot")
```

```
"""  
rag_pipeline.py  
-----  
FR5 + FR7 + FR8 (Q&A, Generation, Citations)  
  
FIXES in this version:  
1. Score threshold – only show sources with score > MIN_SCORE.  
   Low-relevance chunks are retrieved for context but NOT shown as citations.  
   This stops irrelevant pages appearing in the sources panel.  
  
2. No citations on "not found" answers – if the LLM says it could not find  
   the information, sources are cleared so the UI shows no citations.  
  
3. Dynamic citation count – instead of always showing exactly TOP_K sources,  
   only sources that actually contributed to the answer are shown.  
"""  
from __future__ import annotations  
  
import logging  
import textwrap  
import traceback  
from dataclasses import dataclass, field  
from typing import List, Optional  
  
from src.config import (  
    MAX_NEW_TOKENS, PHI2_MODEL_NAME, TEMPERATURE,  
    TOP_K_RESULTS, VECTORSTORE_DIR,  
)  
from src.vector_store import ComplianceVectorStore  
  
logger = logging.getLogger(__name__)  
  
# Only show a source as a citation if its score is above this threshold.  
# Scores below this mean the chunk wasn't actually relevant to the question.  
MIN_CITATION_SCORE = 0.15
```

```
# Phrases that indicate the LLM could not find the answer
NOT_FOUND_PHRASES = [
    "could not find",
    "not found in",
    "not available in",
    "no information",
    "not mentioned",
    "not specified",
    "outside the scope",
    "not covered",
    "i don't know",
    "i do not know",
]

@dataclass
class CitedSource:
    document_name: str
    page_number: int
    excerpt: str
    relevance_score: float

@dataclass
class RAGResponse:
    question: str
    answer: str
    sources: List[CitedSource] = field(default_factory=list)
    has_context: bool = True
    error: str = ""

    def format_sources(self) -> str:
        if not self.sources:
            return "No sources found."
        lines = []
        for i, s in enumerate(self.sources, 1):
            lines.append(
                f"[{i}] {s.document_name} | Page {s.page_number}\n"
                f"    ...{s.excerpt}..."
            )
        return "\n".join(lines)
```

```
def _answer_is_not_found(answer: str) -> bool:
    """Return True if the answer indicates the info wasn't in the documents."""
    a = answer.lower()
    return any(phrase in a for phrase in NOT_FOUND_PHRASES)

class ComplianceRAGPipeline:

    def __init__(
        self,
        vectorstore: Optional[ComplianceVectorStore] = None,
        top_k: int = TOP_K_RESULTS,
        model_name: str = PHI2_MODEL_NAME,
        max_new_tokens: int = MAX_NEW_TOKENS,
        temperature: float = TEMPERATURE,
    ):
        self.top_k = top_k
        self.model_name = model_name
        self.max_new_tokens = max_new_tokens
        self.temperature = temperature
        self.vectorstore = vectorstore or
ComplianceVectorStore(persist_dir=VECTORSTORE_DIR)
        self._llm = None
        self._llm_error = ""

    def _get_llm(self):
        if self._llm is not None:
            return self._llm
        if self._llm_error:
            raise RuntimeError(self._llm_error)
        try:
            from src.llm_engine import get_llm
            self._llm = get_llm(
                model_name=self.model_name,
                max_new_tokens=self.max_new_tokens,
                temperature=self.temperature,
            )
            return self._llm
        except Exception as exc:
            self._llm_error = str(exc)
            raise
```

```
def _fallback_answer(self, results) -> str:
    lines = ["⚠️ Language model unavailable – showing matching policy
excerpts:\n"]
    for i, (doc, score) in enumerate(results, 1):
        lines.append(
            f"Excerpt {i} (Page {doc.metadata.get('page','?')}):\n"
            f"{doc.page_content.strip()}\n"
        )
    return "\n".join(lines)

def query(self, question: str) -> RAGResponse:
    question = question.strip()
    if not question:
        return RAGResponse(question=question,
                            answer="Please enter a question.",
                            has_context=False)

    # — Step 1: Retrieve —————
    try:
        results = self.vectorstore.similarity_search(question, k=self.top_k)
    except RuntimeError as exc:
        return RAGResponse(
            question=question, has_context=False,
            answer="⚠️ No documents indexed yet. Upload a PDF first.",
            error=str(exc),
        )

    if not results:
        return RAGResponse(
            question=question, sources=[],
            answer="I could not find any relevant information in the policy
documents.",
        )

    # — Step 2: Build context (use ALL retrieved chunks for LLM context) —
    # But only mark HIGH-SCORE chunks as citations for the user to see.
    context_parts = []
    all_sources = [] # all retrieved, for context
    cited_sources = [] # only high-score, shown as citations

    for rank, (doc, score) in enumerate(results, 1):
        meta = doc.metadata
```

```
excerpt = textwrap.shorten(doc.page_content, width=200,
placeholder="...")

context_parts.append(
    f"[Source {rank}]: {meta.get('source', 'unknown')},"
    f" Page {meta.get('page', '?')}]\\n{doc.page_content}"
)

source = CitedSource(
    document_name=meta.get("source", "Unknown"),
    page_number=meta.get("page", 0),
    excerpt=excerpt,
    relevance_score=score,
)
all_sources.append(source)

# Only cite if score is above threshold
if score >= MIN_CITATION_SCORE:
    cited_sources.append(source)

context = "\\n\\n---\\n\\n".join(context_parts)

# — Step 3: Generate answer —————
try:
    from src.llm_engine import build_prompt
    prompt = build_prompt(context=context, question=question)
    llm = self._get_llm()
    answer = llm.generate(prompt)

    if not answer or not answer.strip():
        answer = self._fallback_answer(results)

except Exception as exc:
    logger.error("LLM failed: %s\\n%s", exc, traceback.format_exc())
    return RAGResponse(
        question=question,
        answer=self._fallback_answer(results),
        sources=cited_sources,
        has_context=True,
        error=f"{type(exc).__name__}: {exc}",
    )

# — Step 4: Decide whether to show citations —————
```

GenAI Banking Policy & Compliance Copilot

```
# If the answer says "not found", clear citations – nothing relevant was
found.

if _answer_is_not_found(answer):
    final_sources = []
else:
    final_sources = cited_sources

return RAGResponse(
    question=question,
    answer=answer,
    sources=final_sources,
    has_context=True,
)

def ingest_pdf(self, pdf_path, chunk_size=None, chunk_overlap=None):
    from pathlib import Path
    from src.config import CHUNK_OVERLAP, CHUNK_SIZE
    from src.document_processor import load_and_chunk_pdf
    chunks = load_and_chunk_pdf(
        Path(pdf_path),
        chunk_size = chunk_size or CHUNK_SIZE,
        chunk_overlap = chunk_overlap or CHUNK_OVERLAP,
    )
    self.vectorstore.add_documents(chunks)
    return len(chunks)
```

```
"""
evaluate_rouge.py
-----
Offline evaluation of the Compliance Copilot using ROUGE scores.
No internet, no API key, no BERTScore download needed.

Install: pip install rouge-score
Run:     python evaluate_rouge.py

What ROUGE measures:
    ROUGE-1 = overlap of individual words between answer and reference
    ROUGE-2 = overlap of word pairs (bigrams)
    ROUGE-L = longest common subsequence (word order similarity)
```

